SOFTWARE ENGINEERING PROJECT SPECIFICATION

PALMYRA

Modern Full-Stack Online Auction & Marketplace Platform

Project Title	PALMYRA — Online Auction & Marketplace
System Architecture	SvelteKit 2 + Svelte 5 + Supabase BaaS (PostgreSQL, Auth, Storage)
Primary Innovation	Canonical 4:3 Aspect Ratio Pipeline with In-Browser Canvas Cropper
Branch & Environment	`sellerproduction` | Production Ready
Release Date	November 2026

1. Project Title

PALMYRA — An Ultra-Responsive, Cloud-Native Online Auction and Multi-Vendor Marketplace Platform built with SvelteKit 2, Svelte 5 Runes, Tailwind CSS v4, and Supabase Backend-as-a-Service.

2. Introduction

Modern e-commerce requires dynamic, fair pricing mechanisms alongside rapid visual engagement. Conventional classifieds and fixed-price marketplaces lack dynamic price discovery, whereas legacy auction systems suffer from clunky user interfaces, full-page reloads, distorted images, and poor mobile accessibility.

PALMYRA resolves these deficiencies through a modern, cloud-native architecture combining SvelteKit 2 (with Svelte 5's new reactivity runes) on the frontend and Supabase (PostgreSQL 15, Auth with SSR cookies, and Storage CDN) on the backend.

A flagship engineering highlight of PALMYRA is its **Zero-Distortion Media Pipeline**: by enforcing a canonical **4:3 aspect ratio** across every touchpoint (thumbnails, gallery cards, and hero viewports) via an interactive client-side canvas cropper and WebP compressor, the platform guarantees zero cumulative layout shifts (CLS) and snappy mobile performance.

3. Objectives

- **Real-Time Dynamic Marketplace:** Implement competitive live auctions featuring starting bids, Buy-It-Now instant checkouts, reserve thresholds, and dynamic countdown timers.
- **Streamlined "Start Selling" Flow:** Facilitate rapid listing creation with return-URL preservation across authentication boundaries, cascading categories, and tag indexing.
- **Standardized 4:3 Media Uniformity:** Eradicate image stretching, letterboxing, and layout jumping by constraining all uploads to 4:3 via an interactive canvas cropper.
- **Mobile-First Ergonomics:** Guarantee touch targets ≥ 44px, sticky bid bars on mobile viewports, zero horizontal overflow, and responsiveness down to 360px widths.
- **Zero-Trust Security & Row Level Security:** Implement PostgreSQL RLS policies that isolate user data, restrict mutations to owners, and enforce cookie session validation.

4. Modules Overview

- **Authentication & Routing Guard Module:** Manages session cookies, OAuth code exchanges, protected route barriers, and return-URL navigation.
- **Seller & Listing Creation Module:** Powers a multi-step listing wizard with cascading category taxonomy, condition grading, price rules, and slug generation.

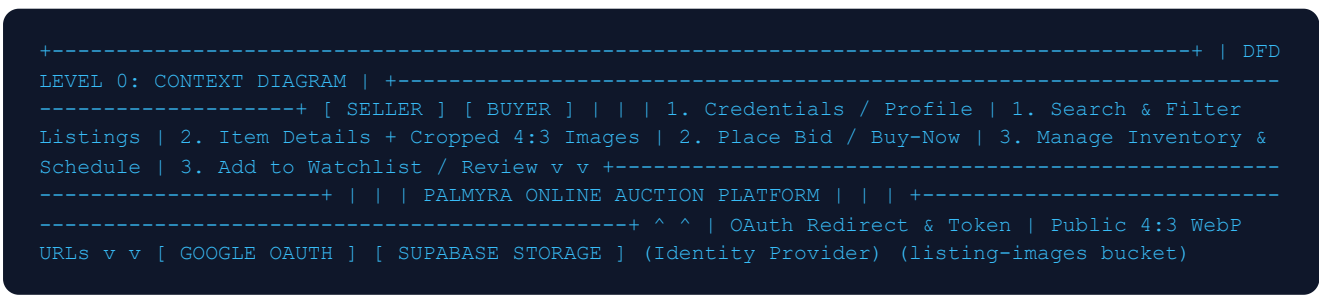
- **Media Pipeline & Storage Module:** Direct camera capture, multi-image upload queue, interactive 4:3 canvas cropper, WebP compression, reordering, and Supabase Storage CDN distribution.
- **Marketplace Discovery & Live Feeds:** Live PostgreSQL queries for active auctions, multi-parameter search, category filtering, sorting, and pagination.
- **Listing Details & Showcase:** 4:3 gallery view with thumbnail carousel, live countdown, view count incrementor, and verified seller info.
- **Auction Bidding & Watchlist:** Bid placement validation, Buy-It-Now execution, and user watchlisting.
- **User Profile & Reputation:** Seller profiles, review score calculation, avatar management, and ratings.

5. Technologies Used

Layer	Technology	Role & Significance
Frontend Framework	SvelteKit 2 + Svelte 5	Runes reactivity (`$state`, `$derived`, `$effect`). Hybrid SSR/CSR ensures instant first-contentful paint with minimal client JavaScript.
Type System	TypeScript 6.0	Full-stack end-to-end type safety for database models, API responses, form actions, and component props.
Styling & Icons	Tailwind CSS v4 + Lucide	Next-generation CSS engine with frosted glass aesthetic, dark/light theme switching, and accessible SVG iconography.
Database & Engine	PostgreSQL 15 (Supabase)	Relational integrity, foreign key cascades, check constraints, UUID primary keys, and PL/pgSQL triggers.
Security & Auth	Supabase Auth + SSR Cookies	Server-side session validation via HTTP-only cookies, OAuth (Google), and Row-Level Security (RLS) enforcement.
Cloud Storage	Supabase Storage (CDN)	Public bucket `listing-images` with authenticated uploads and public CDN delivery.

6. Data Flow Diagrams (DFD)

6.1 DFD Level 0 — Context Diagram



6.2 DFD Level 1 — Subsystem Decomposition

```
+-------------------------------------------------------------------------------------------+ | DFD LEVEL 1: SUBSYSTEM DIAGRAM | +-------------------------------------------------------------------------------------------
----------------------+ [User] ---> (1.0 Auth & Session) <---> [auth.users] / [profiles] | v (Authenticated Context) [Seller] -> (2.0 Listing Publication) ----> [listings] & [listing_tags] | +-> (3.0 Media Cropper & Upload) -> [Supabase Storage] | v (image_url) [listing_images] [Buyer] -
-> (4.0 Browse & Search) <------ [listings] + [listing_images] | +-> (5.0 Bidding Engine) -------
> [bids] & [bid_history] | +-> (6.0 Watchlist Manager) ----> [watchlist]
```

6.3 DFD Level 2 — Seller Listing Creation & Media Flow

- **2.1 Camera / File Picker:** User triggers file selector or native mobile camera (`capture="environment"`).
- **2.2 Interactive 4:3 Canvas Cropper:** Image loaded into HTML5 Canvas; user zooms (1x-5x) and pans.
- **2.3 Offscreen WebP Export:** Canvas outputs 800x600 WebP blob at 0.85 compression quality.
- **2.4 Concurrent Storage Streaming:** Blobs streamed in parallel to `listing-images/{uid}/{listing_id}/{uuid}.webp`.
- **2.5 Server Transaction:** POST action writes atomically to `listings`, `listing_images`, and `listing_tags`.

7. Entity-Relationship (ER) Diagram

```
+-------------------------------------------------------------------------------------------+ | ENTITY-RELATIONSHIP DIAGRAM | +-------------------------------------------------------------------------------------------
----------------------+ [auth.users] 1 -------- 1 [profiles] | 1 | | has many v * [categories] 1
--- * [subcategories] 1 --- * [listings] 1 --- * [listing_images] | | | +------- * [listing_tags]
| +---------- * [watchlist] * --- 1 [profiles] | +---------- * [bids] 1 --- * [bid_history] | | |
v | [profiles] (bidder) | +---------- * [reviews] * --- 1 [profiles]
```

7.1 Cardinality & Integrity Rules

- `profiles` (1) → `listings` (*): One seller creates multiple listings; deletion cascades.
- `categories` (1) → `subcategories` (*): Taxonomy hierarchy; cascading foreign key.
- `listings` (1) → `listing_images` (*): 1-to-many relationship with order index and primary flag.
- `listings` (1) → `listing_tags` (*): 1-to-many tag index for search filtering.
- `listings` (1) → `bids` (*): Auction bids placed by buyers; records audit trail in `bid_history`.

8. Database Design & Schema Specification

Table: listings

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY, gen_random_uuid()	Unique listing identifier
seller_id	uuid	FK → profiles(id) ON DELETE CASCADE	Seller account ID
subcategory_id	uuid	FK → subcategories(id)	Category taxonomy linkage
title	text	NOT NULL	Listing headline (max 120 chars)
slug	text	NOT NULL, UNIQUE	URL slug with unique random suffix
description	text	NOT NULL	Markdown/text item details
condition	text	NOT NULL	Enum: brand_new, like_new, open_box, etc.
starting_price	numeric	NOT NULL, CHECK (> 0)	Opening bid in INR (≥ 1)
current_price	numeric	NOT NULL	Current highest bid or opening price
buy_it_now_price	numeric	NULLABLE	Instant buyout price
reserve_price	numeric	NULLABLE	Minimum hidden reserve
quantity	int4	DEFAULT 1, CHECK (≥ 1)	Inventory lot count
status	text	DEFAULT 'active'	Enum: active, ended, sold, cancelled
start_date	timestamptz	NOT NULL	Auction start time
end_date	timestamptz	NOT NULL	Auction closing deadline
views	int4	DEFAULT 0	Accumulated page views
created_at	timestamptz	DEFAULT now()	Creation timestamp

Table: listing_images

Column	Type	Constraints	Description
`id`	uuid	PRIMARY KEY	Unique image record ID
`listing_id`	uuid	FK → listings(id) ON DELETE CASCADE	Associated listing
`image_url`	text	NOT NULL	Public CDN URL (4:3 WebP)
`is_primary`	bool	DEFAULT false	Flag for primary card/hero showcase
`display_order`	int4	DEFAULT 0	Ordering sequence index
`created_at`	timestamptz	DEFAULT now()	Upload timestamp

Table: profiles

Column	Type	Constraints	Description
`id`	uuid	PRIMARY KEY → auth.users(id)	Auth user account ID
`username`	text	NOT NULL, UNIQUE	Display handle
`full_name`	text	NULLABLE	Full user name
`avatar_id`	text	NULLABLE	Avatar path in storage bucket
`bio`	text	NULLABLE	Biography summary
`rating`	numeric	DEFAULT 0.0	Aggregated rating score (0.0 - 5.0)
`total_reviews`	int4	DEFAULT 0	Total review counter

9. Functionalities of Each Module

9.1 Authentication & Navigation Guards

Session tokens are managed using HTTP-only cookies verified on every request by `src/hooks.server.ts`. If an unauthenticated user clicks "Start Selling", the system routes them to `/auth/signin?next=/sell`, preserving their intended destination across both password-based and OAuth Google sign-ins.

9.2 Seller & Listing Creation Wizard

Located at `/sell`, this multi-step form guides sellers through:

- **Step 1 (Photos):** Multi-upload with native camera option, canvas cropping, and upload progress indicators.
- **Step 2 (Details):** Item title, description, cascading category → subcategory selector, condition selection buttons, and tags.

- **Step 3 (Pricing & Dates):** Starting price, optional Buy-It-Now price, optional reserve price, inventory quantity, and start/end datetime selectors.

9.3 4:3 Image Cropping & Compression Pipeline

Every photo uploaded must conform to the **4:3 aspect ratio**. The `ImageCropper` provides an interactive viewport with zoom and pan controls. Upon confirmation, the offscreen canvas compresses the image into an 800×600 WebP blob at 85% quality. Files are streamed in parallel to Supabase Storage, and public URLs are assigned.

9.4 Marketplace Discovery & Browse

The homepage (`/`) and browse page (`/listings`) execute live PostgreSQL queries against Supabase:

```
SELECT * FROM listings WHERE status = 'active' AND start_date <= now() AND end_date >= now()
ORDER BY current_price DESC;
```

10. Future Scope

- **Supabase Realtime WebSocket Bids:** Live bid stream updates pushed to all active viewing clients without page refreshes.
- **Escrow Payment Gateway:** Automated payment hold and release upon confirmed delivery via Stripe Connect.
- **AI-Assisted Item Grading:** Computer vision model to evaluate listing photos against declared conditions and flag counterfeit items.
- **Automated Settlement Cron (pg_cron):** Automatic state transitions from 'active' to 'sold'/'ended' upon expiration, triggering automated winner notifications.

11. Execution of the Project

11.1 Prerequisites

- Node.js v20.x or higher
- pnpm v9 or v10
- Supabase project instance with PostgreSQL database and Storage enabled

11.2 Environment Configuration

```
# .env.local PUBLIC_SUPABASE_URL=https://<your-project-id>.supabase.co
PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_<your-publishable-key>
```

11.3 Database Setup

Execute the full migration script located at `docs/supabase-setup.sql` in the Supabase SQL Editor. Ensure the `listing-images` bucket is set to Public with authenticated upload RLS policies.

11.4 Running Locally

```
# Install all dependencies pnpm install # Run development server pnpm dev # Available at http://localhost:5173
```

11.5 Building for Production

```
# Run TypeScript and Svelte diagnostics check pnpm check # Create production build pnpm build # Preview build locally pnpm preview
```